CSE 2006 - Programming in Java

Course Type: LP Credits: 3

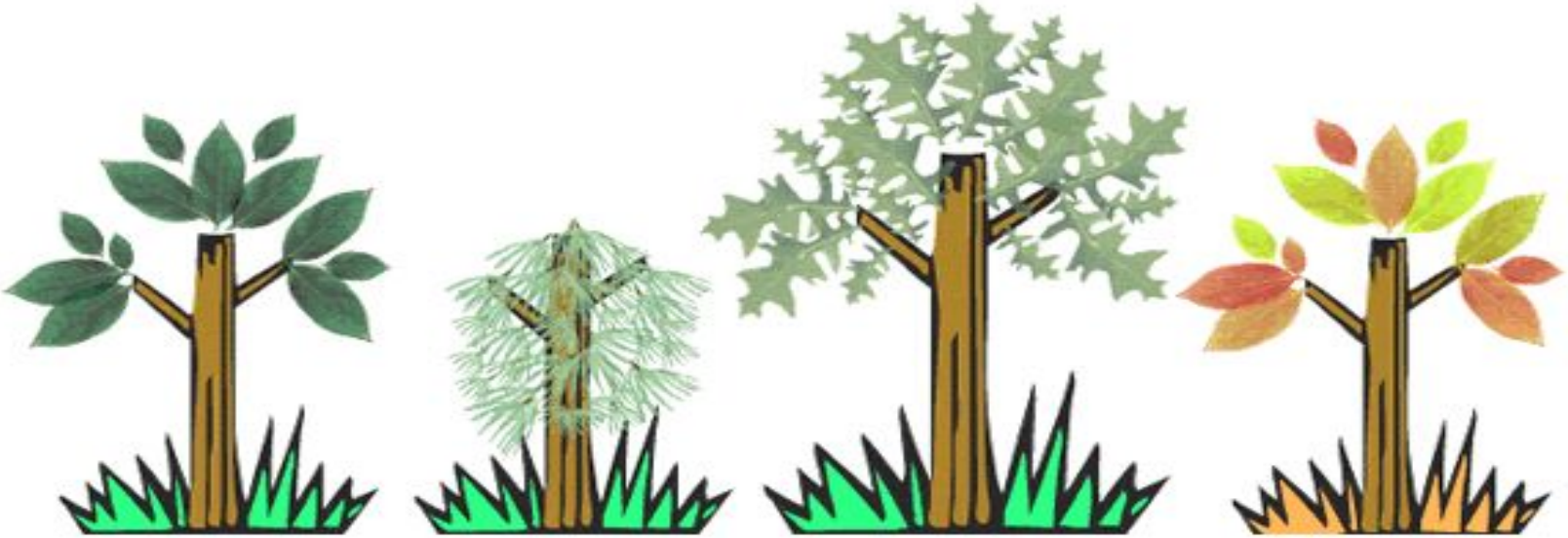
Prepared by
C.S Rajpoot
School of Computing Science and Engineering
VIT Bhopal University

Unit-2 Java Object-oriented Programming

- **Java OOP (Basics)**
- Java Class and Objects, Java Methods, Java Constructor, Java Strings, Java Access Modifiers, Java this keyword, Java final keyword, Java Recursion, Java instance of Operator, Java Single Class and Anonymous Class, Java enum Class
- **Java OOP (Inheritance & Polymorphism)**
- Java Inheritance, Java Method Overriding, Java super Keyword, Abstract Class & Method, Java Interfaces, Java Polymorphism (overloading & overriding), Java Encapsulation
- **Java OOP (Other types of classes)**
- Nested & Inner Class, Java Static Class, Java Anonymous Class, Java Singleton, Java enum Class, Java enum Constructor, Java enum String, Java Reflection

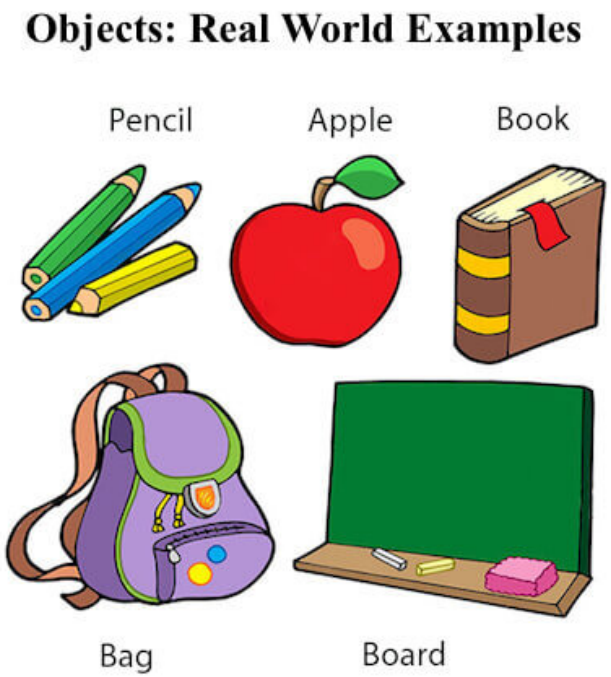
What do you understand?

Class	Tree
Properties (data)	`height` `color`
Behaviors (methods)	`changeColor()`



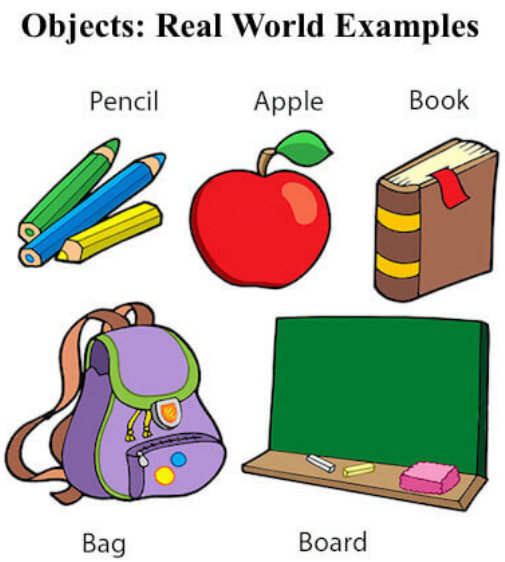
Java Class and Objects

- An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.
- **What is an object in Java?**
- An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

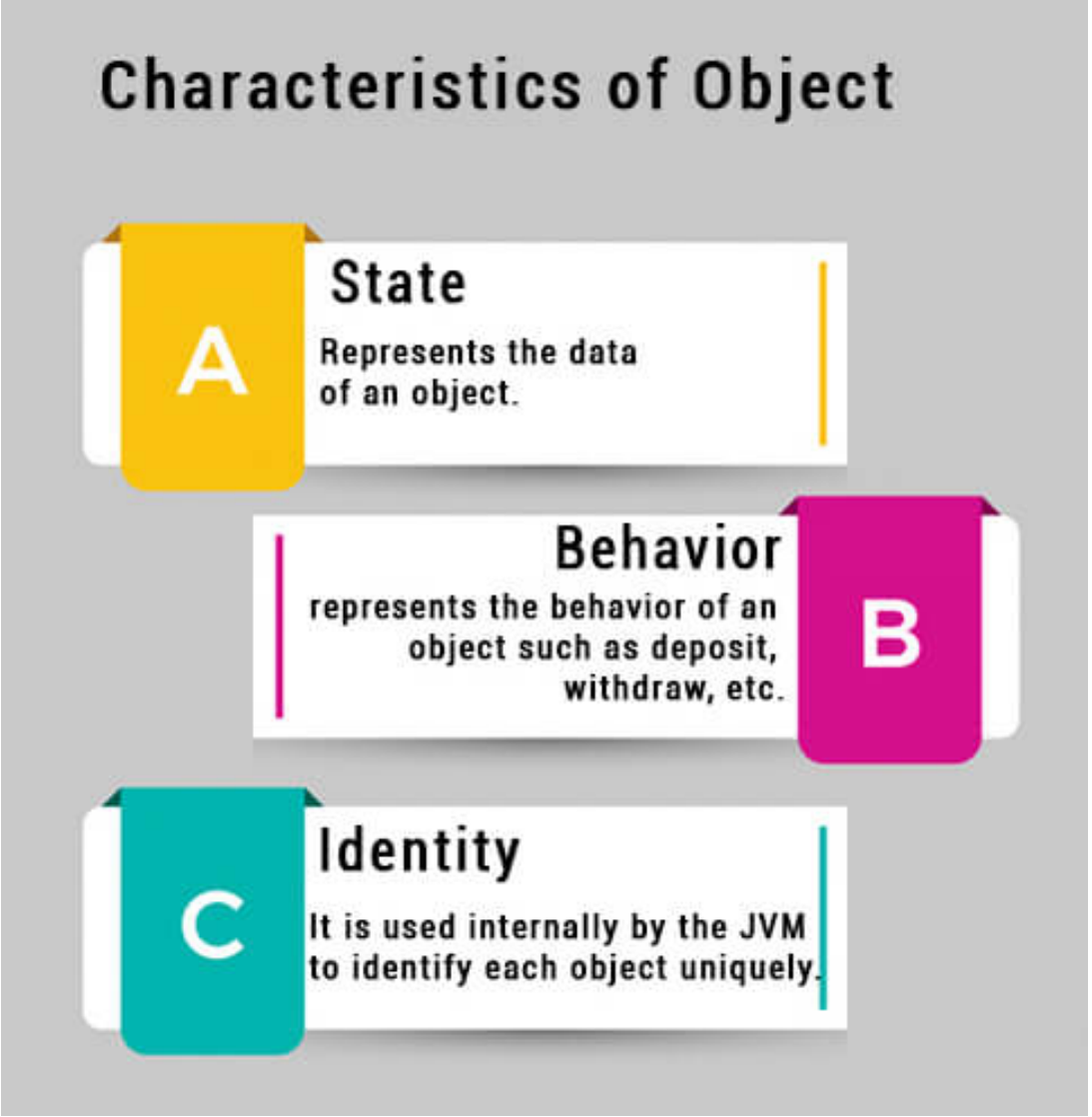


Java Class and Objects

- **An object has three characteristics:**
- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.



Java Class and Objects

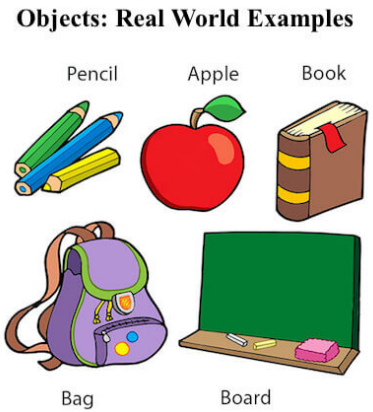


Java Class and Objects

- For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.
- **An object is an instance of a class.** A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

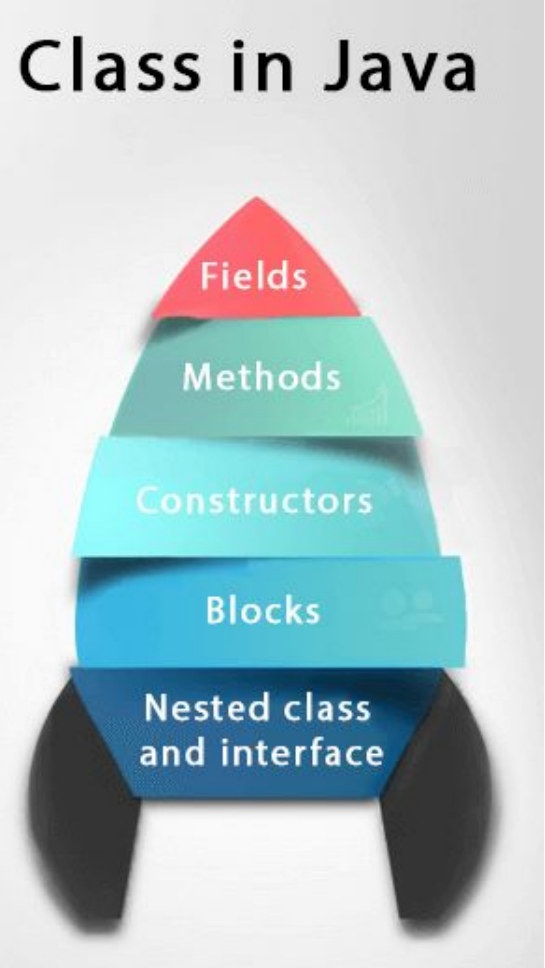
Object Definitions:

- An object is *a real-world entity*.
- An object is *a runtime entity*.
- The object is *an entity which has state and behavior*.
- The object is *an instance of a class*.



Java Class and Objects

- **What is a class in Java**
- A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.
- A class in Java can contain:
  - **Fields**
  - **Methods**
  - **Constructors**
  - **Blocks**
  - **Nested class and interface**



Syntax to declare a class:

```
class <class_name>
{
   field;
   method;
}
```

Java Class and Objects

- **Instance variable in Java**
- A variable which is created **inside the class but outside the method is known as an instance variable.** Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.
- **new keyword in Java**
- The new keyword is used to **allocate memory at runtime.**
- All objects get memory in Heap memory area.
- **Method in Java**
- In Java, a method is like a function which is used to expose the behavior of an object.
- **Advantage of Method**
  - **Code Reusability and Code Optimization**

Java Class and Objects

- **Object and Class Example: main within the class**
- created a Student class which has two data members id and name. We are creating the object of the Student class by new keyword and printing the object's value. Here, we are creating a main() method inside the class.
- ***File: Student.java***

```
class Student{        //defining fields
 int id;     //field or data member or instance variable
 String name;  //creating main method inside the Student class
 public static void main(String args[]){
  //Creating an object or instance
  Student s1=new Student();//creating an object of Student
  //Printing values of the object
  System.out.println(s1.id);//accessing member through reference variable
  System.out.println(s1.name);
 }
}
```

Output:
0
null

Java Class and Objects

- **Object and Class Example: main outside the class**
- In real time development, we create classes and use it from another class. It is a better approach than previous one.
- Let's see a simple example, where we are having main() method in another class.

- We can have multiple classes in different Java files or single Java file. If you define multiple classes in a single Java source file, it is a good idea to save the file name with the class name which has main() method.

Java Class and Objects

- **3 Ways to initialize object**
- There are 3 ways to initialize object in Java.

1. By reference variable
2. By method
3. By constructor

Java Class and Objects

- **1) Object and Class Example: Initialization through reference**
- Initializing an object means storing data into the object. Let's see a simple example where we are going to initialize the object through a reference variable.
- *File: TestStudent2.java*

```
class Student{
 int id;
 String name;
}
class TestStudent2{
 public static void main(String args[]){
  Student s1=new Student();
  s1.id=101;
  s1.name="Sonoo";
  System.out.println(s1.id+" "+s1.name);//printing members with a white space
 }
}
```

Output:
101 Sonoo

Java Class and Objects

- We can also create multiple objects and store information in it through reference variable.
- ***File: TestStudent3.java***

```
class Student{
 int id;
 String name;
}
class TestStudent3{
 public static void main(String args[]){
  //Creating objects
  Student s1=new Student();
  Student s2=new Student();
  //Initializing objects
  s1.id=101;
  s1.name="Sonoo";
  s2.id=102;
  s2.name="Amit";
```

```
//Printing data
  System.out.println(s1.id+" "+s1.name);
  System.out.println(s2.id+" "+s2.name);
 }
}
```

Output:

101 Sonoo

102 Amit

- **2) Object and Class Example: Initialization through method**
- In this example, we are creating the two objects of Student class and initializing the value to these objects by invoking the insertRecord method. Here, we are displaying the state (data) of the objects by invoking the displayInformation() method.

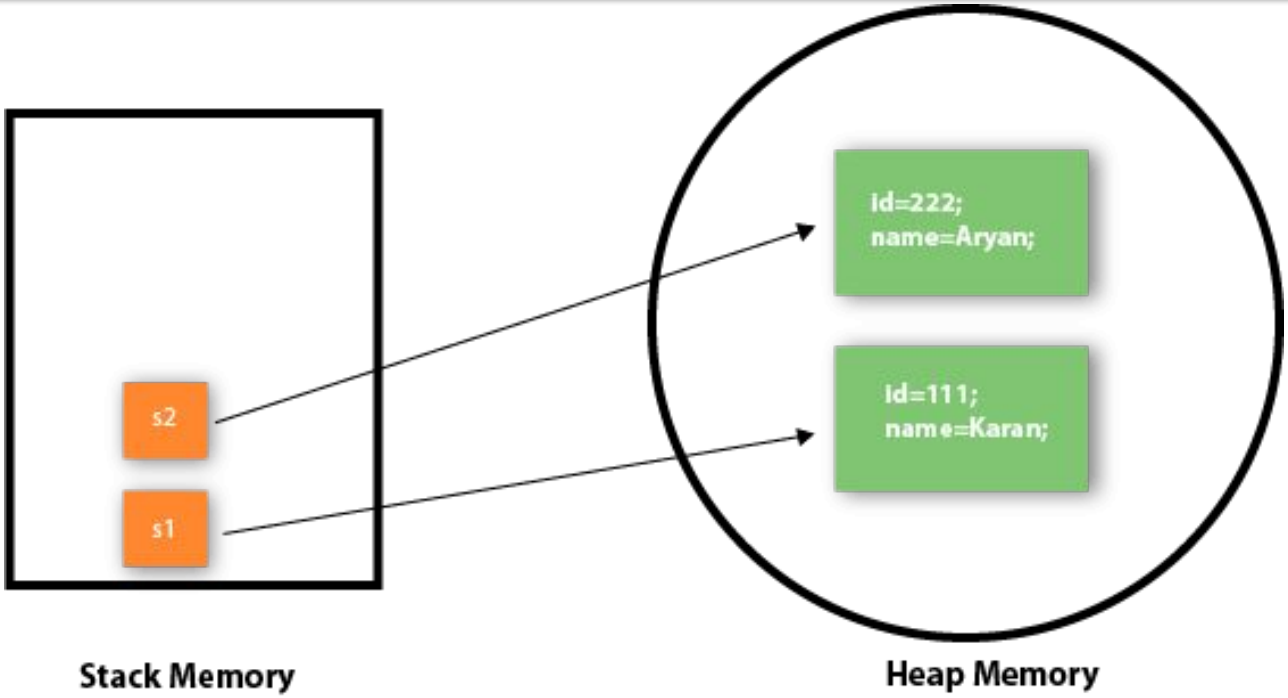
Java Class and Objects

- ***File: TestStudent4.java***

```
class Student{
 int rollno;
 String name;
 void insertRecord(int r, String n){
  rollno=r;
  name=n;   }
 void displayInformation() {System.out.println(rollno+" "+name);}  }
class TestStudent4{
 public static void main(String args[]){
  Student s1=new Student();
  Student s2=new Student();
  s1.insertRecord(111,"Karan");
  s2.insertRecord(222,"Aryan");
  s1.displayInformation();
  s2.displayInformation();
 }  }
```

Output:
111 Karan
222 Aryan

Java Class and Objects



Object gets the memory in heap memory area. The reference variable refers to the object allocated in the heap memory area. Here, s1 and s2 both are reference variables that refer to the objects allocated in memory.

Java Class and Objects

- **3) Object and Class Example: Initialization through a constructor**

- A constructor in Java is a **special method** that is used to initialize objects. The constructor is called when an object of a class is created. It can be used to set initial values for object attributes

Java Class and Objects

Object and Class Example: Employee

File: TestEmployee.java

```
class Employee{
   int id;
   String name;
   float salary;
   void insert(int i, String n, float s)
{
      id=i;
      name=n;
      salary=s;
   }
   void display(){System.out.println(id+" "+name+" "+salary);}
}
```

```
public class TestEmployee {
public static void main(String[] args) {
   Employee e1=new Employee();
   Employee e2=new Employee();
   Employee e3=new Employee();
   e1.insert(101,"ajeet",45000);
   e2.insert(102,"irfan",25000);
   e3.insert(103,"nakul",55000);
   e1.display();
   e2.display();
   e3.display();
}
}
```

Output:

```
101   ajeet   45000.0
102   irfan   25000.0
103 nakul 55000.0
```

Java Class and Objects

- **Object and Class Example: Rectangle**
- *File: TestRectangle1.java*

```
class Rectangle{
 int length;
 int width;
 void insert(int l, int w){
  length=l;
  width=w;
 }
 void calculateArea()
        {System.out.println(length*width);}
}
```

```
class TestRectangle1{
 public static void main(String args[]){
  Rectangle r1=new Rectangle();
  Rectangle r2=new Rectangle();
  r1.insert(11,5);
  r2.insert(3,15);
  r1.calculateArea();
  r2.calculateArea();
}
}
```

Output:

55

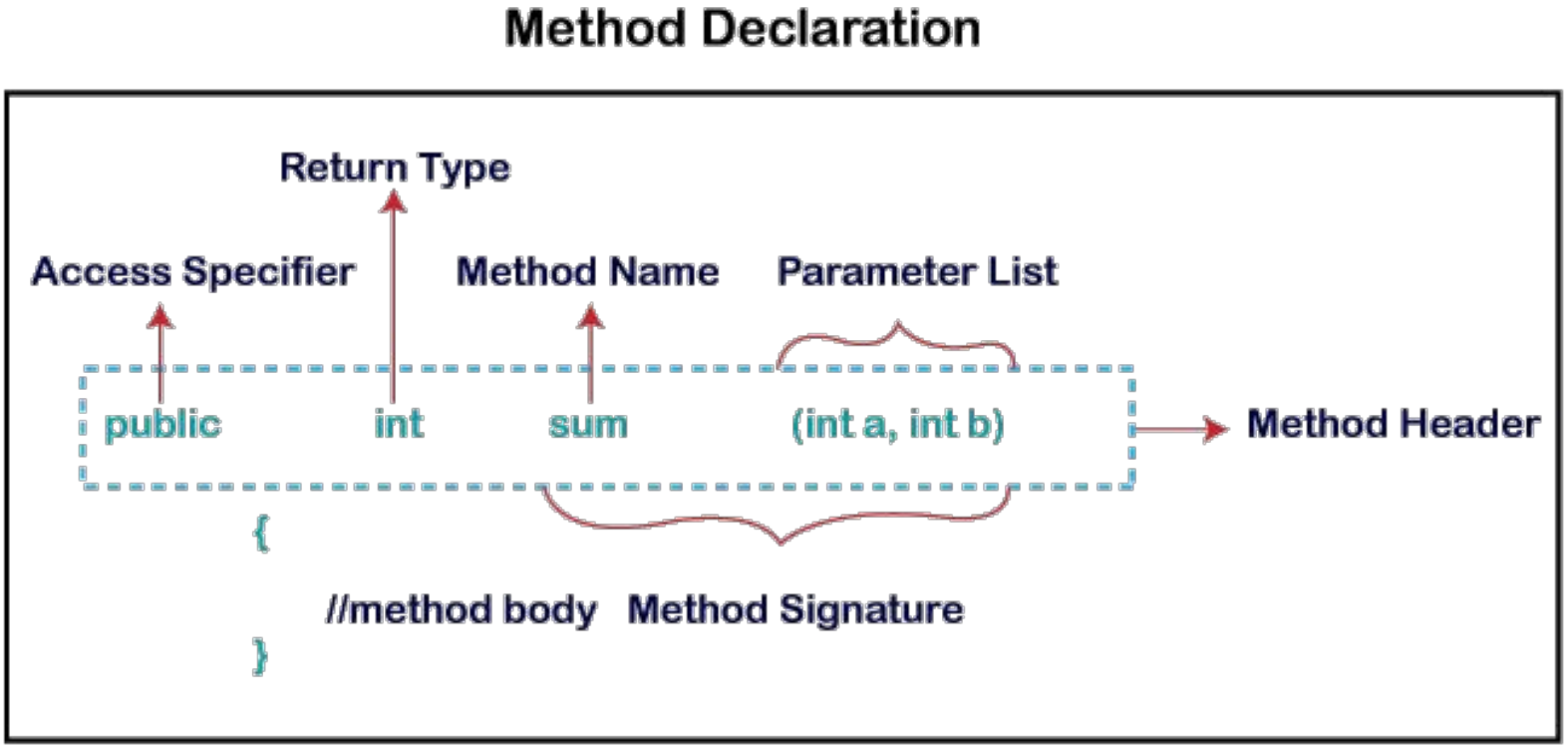
45

Java method

- In general, a **method** is a way to perform some task. Similarly, the **method in Java** is a collection of instructions that performs a specific task. It provides the reusability of code. We can also easily modify code using **methods**. In this section, we will learn **what is a method in Java, types of methods, method declaration,** and **how to call a method in Java**.
- **What is a method in Java?**
- A **method** is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation. It is used to achieve the **reusability** of code. We write a method once and use it many times. We do not require to write code again and again. It also provides the **easy modification** and **readability** of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.
- The most important method in Java is the **main()** method. If you want to read more about the main() method.

Java method

- **Method Declaration**
- The method declaration provides information about method attributes, such as visibility, return-type, name, and arguments. It has six components that are known as **method header**, as we have shown in the following figure.



Java method

- **Method Signature:** Every method has a method signature. It is a part of the method declaration. It includes the **method name** and **parameter list**.
- **Access Specifier:** Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides **four** types of access specifier:
  - **Public:** The method is accessible by all classes when we use public specifier in our application.
  - **Private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
  - **Protected:** When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
  - **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Java method

- **Return Type:** Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use void keyword.
- **Method Name:** It is a unique name that is used to define the name of a method. It must be corresponding to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be **subtraction().** A method is invoked by its name.
- **Parameter List:** It is the list of parameters separated by a comma and enclosed in the pair of parentheses. It contains the data type and variable name. If the method has no parameter, left the parentheses blank.
- **Method Body:** It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within the pair of curly braces.

Java method

- **Naming a Method**
- While defining a method, remember that the method name must be a **verb** and start with a **lowercase** letter. If the method name has more than two words, the first name must be a verb followed by adjective or noun. In the multi-word method name, the first letter of each word must be in **uppercase** except the first word. For example:
- **Single-word method name:** sum(), area()

Java method

- **Multi-word method name:** areaOfCircle(), stringComparision()
- It is also possible that a method has the same name as another method name in the same class, it is known as **method overloading**.
- **Types of Method**
- There are two types of methods in Java:
  - **Predefined Method**
  - **User-defined Method**
- **Predefined Method**
- In Java, predefined methods are the method that is already defined in the Java class libraries is known as predefined methods. It is also known as the **standard library method** or **built-in method**. We can directly use these methods just by calling them in the program at any point. Some pre-defined methods are **length(), equals(), compareTo(), sqrt(),** etc. When we call any of the predefined methods in our program, a series of codes related to the corresponding method runs in the background that is already stored in the library.

Java method

- Each and every predefined method is defined inside a class. Such as **print()** method is defined in the **java.io.PrintStream** class. It prints the statement that we write inside the method. For example, **print("Java")**, it prints Java on the console.

- Let's see an example of the predefined method.

Demo.java

```
public class Demo
{
public static void main(String[] args)
{
// using the max() method of Math class
System.out.print("The maximum number is: " + Math.max(9,7));
}
}
```

Output:

The maximum number is: 9

Java method

- **User-defined Method**
- The method written by the user or programmer is known as **a user-defined** method. These methods are modified according to the requirement.
- How to Create a User-defined Method
- Let's create a user defined method that checks the number is even or odd. First, we will define the method.

- //user defined method

```
public static void findEvenOdd(int num)
{
//method body
if(num%2==0)
System.out.println(num+" is even");
else
System.out.println(num+" is odd");
}
```

Java method

- **How to Call or Invoke a User-defined Method**
- Once we have defined a method, it should be called. The calling of a method in a program is simple. When we call or invoke a user-defined method, the program control transfer to the called method.

```
import java.util.Scanner;
public class EvenOdd
{
public static void main (String args[])
{
//creating Scanner class object
Scanner scan=new Scanner(System.in);
System.out.print("Enter the number: ");
//reading value from the user
int num=scan.nextInt();
//method calling
findEvenOdd(num);
}
```

Java method

- Let's combine both snippets of codes in a single program and execute it.
- **EvenOdd.java**

```
import java.util.Scanner;
public class EvenOdd
{
public static void main (String args[])
{
//creating Scanner class object
Scanner scan=new Scanner(System.in);
System.out.print("Enter the number: ");
//reading value from user
int num=scan.nextInt();
//method calling
findEvenOdd(num);
}
```

```
//user defined method
public static void findEvenOdd(int num)
{
//method body
if(num%2==0)
System.out.println(num+" is even");
else
System.out.println(num+" is odd");
}
}
```

Output:

Enter the number: 12

12 is even

Java method

- **Addition.java**

```
public class Addition
{
public static void main(String[] args)
{
int a = 19;
int b = 5;
//method calling
int c = add(a, b);   //a and b are actual parameters
System.out.println("The sum of a and b is= " + c);
}
```

```
//user defined method
public static int add(int n1, int n2)
//n1 and n2 are formal parameters
{
int s;
s=n1+n2;
return s; //returning the sum
}
}
```

Output:

The sum of a and b is= 24

Java method

- **Static Method**
- A method that has static keyword is known as static method. In other words, a method that belongs to a class rather than an instance of a class is known as a static method. We can also create a static method by using the keyword **static** before the method name.
- **The main advantage of a static method is that we can call it without creating an object.**
- It can access static data members and also change the value of it. It is used to create an instance method. It is invoked by using the class name. The best example of a static method is the **main()** method.

Java method

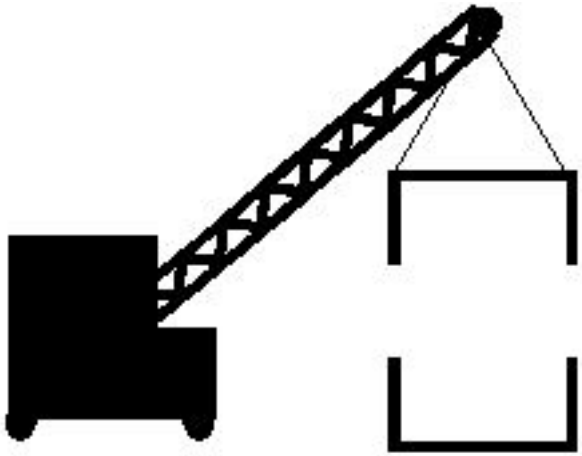
- **Example of static method**
- **Display.java**

```
public class Display
{
public static void main(String[] args)
{
show();
}
static void show()
{
System.out.println("It is an example of static method.");
}
}
```

Output:
It is an example of a static method.

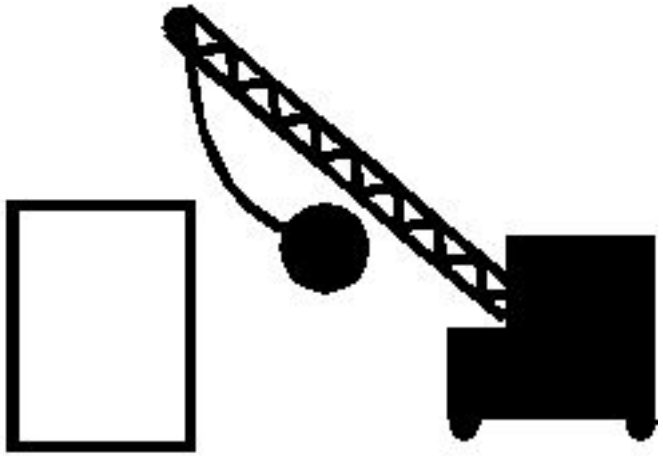
More details:
https://www.javatpoint.com/method-in-java

What do you understand?



Constructor

```
MyClass *MyObjPtr = new MyClass();
```



Destructor

```
delete MyObjPtr;
```

Constructors

- In Java, a constructor is a block of codes **similar to the method.**
- It is called when an **instance of the class** is created. At the time of **calling constructor, memory for the object is allocated** in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the **new() keyword**, at least one constructor is called.
- It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.
- There are **two types of constructors in Java**:
  - **No-argument constructor, and parameterized constructor.**
- **Note:** It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Constructors

- **Rules for creating Java constructor**
- There are two rules defined for the constructor.
  - **Constructor name must be the same as its class name**
  - **A Constructor must have no explicit return type**
- A Java constructor cannot be **abstract, static, final, and synchronized**

- Note: We can use access modifiers while declaring a constructor.
- It controls the object creation. In other words, we can have private, protected, public or default constructor in Java.

Constructors

- **Types of Java constructors**
- **There are two types of constructors in Java:**
  1. **Default constructor (no-argument constructor)**
  2. **Parameterized constructor**

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

Example of default constructor

```
//Java Program to create and call a default constructor
class Bike1{
//creating a default constructor
Bike1()
{  System.out.println("Bike is created"); }
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}  }
```

Output:
Bike is created

Constructors

- **Rule: If there is no constructor in a class, compiler automatically creates a default constructor.**

- **What is the purpose of a default constructor?**
- The default constructor is used to provide the default values to the object like **0, null, etc**., depending on the type.

Constructors

Example of default constructor that displays the default values

```
//Let us see another example of default constructor
//which displays the default values
class Student3{
int id;
String name;
//method to display the value of id and name
void display(){System.out.println(id+" "+name);}

public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the object
s1.display();
s2.display();
}
}
```

Output:

```
0 null
0 null
```

Constructors

- **Parameterized Constructor**
- A constructor which has a specific number of parameters is called a parameterized constructor.
- **Why use the parameterized constructor?**
- The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

- In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

Constructors

```
//Java Program to demonstrate the use of the parameterized constructor.
class Student4{
   int id;
   String name;
   //creating a parameterized constructor
   Student4(int i,String n){
   id = i;
   name = n;
   }
   //method to display the values
   void display(){System.out.println(id+" "+name);}
     public static void main(String args[]){
   //creating objects and passing values
   Student4 s1 = new Student4(111,"Karan");
   Student4 s2 = new Student4(222,"Aryan");
   //calling method to display the values of object
   s1.display();
   s2.display();
  }
}
```

Output:

111 Karan

222 Aryan

```
// parameterized constructor to initialize the dimensions of a box.
class Box {
double width;
double height;
double depth;
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
double volume() // compute and return volume
{   return width * height * depth;  }  }
class BoxDemo7 {
public static void main(String args[]) {
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box(3, 6, 9);
double vol;
vol = mybox1.volume(); // get volume of first box
System.out.println("Volume is " + vol);
vol = mybox2.volume();        // get volume of second box
System.out.println("Volume is " + vol);
}}
```

Output:

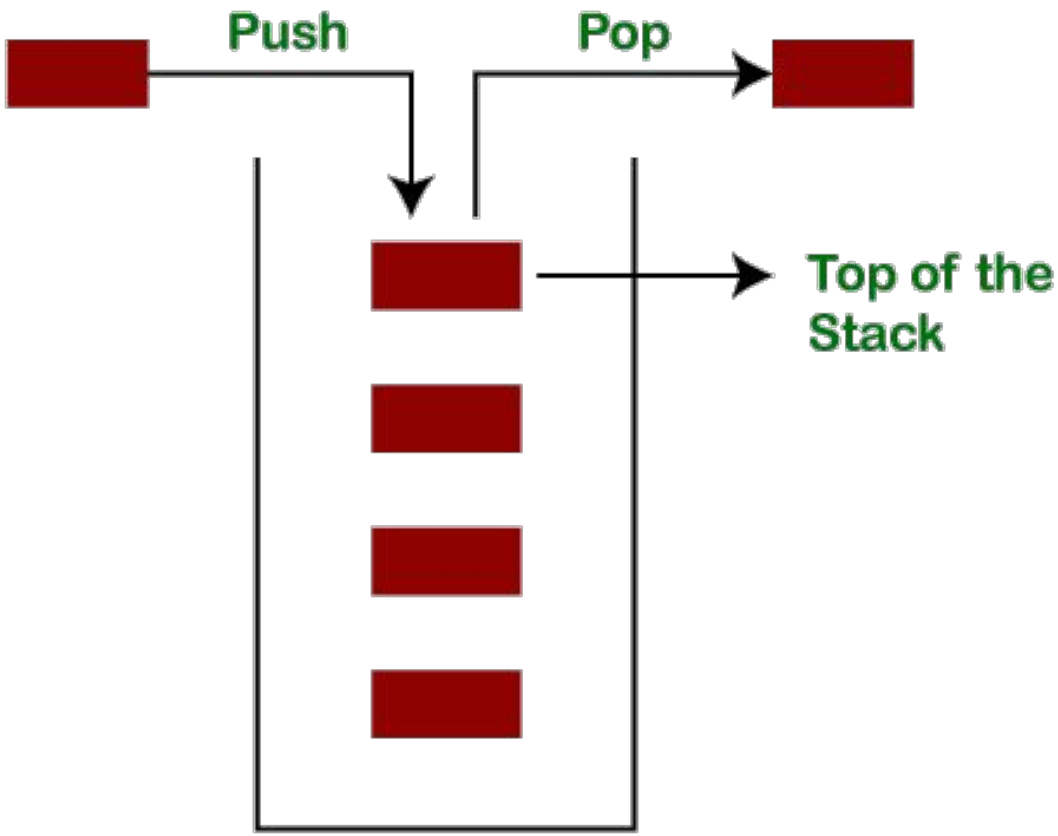
Volume is 3000.0
Volume is 162.0

This Java program defines a class Employee to accept emp_id, emp _name, basic_salary from the user and display the gross_salary.

```
import java.lang.*;
import java.io.*;
class Employee
{
        int emp_id;
        String emp_name;
        float basic_salary;
        Employee(int id, String name, float sal)
        {
                emp_id=id;
                emp_name=name;
                basic_salary=sal;
        }
        void display()
        {
                float da=basic_salary*15/100;
                float hra=basic_salary*10/100;
                float gross_sal=basic_salary+da+hra;
                System.out.println ("Employee Id= "+emp_id);
                System.out.println ("Emplyee Name= "+emp_name);
                System.out.println ("Gross Salary= "+gross_sal);
        }
}
```

```
class q4Employee
{
        public static void main(String args[]) throws IOException
        {
                BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
                System.out.println ("Enter Employee id");
                int id = Integer.parseInt(br.readLine());
                System.out.println ("Enter Employee Name");
                String name = br.readLine();
                System.out.println ("Enter Basic Salary");
                Float sal = Float.parseFloat(br.readLine());
                Employee e = new Employee(id, name, sal);
                e.display();
        }
}
```

Stack Operation



// This class defines an integer stack that can hold 10 values.

```
class Stack {
  int stck[] = new int[10];
  int top;
// Initialize top-of-stack
Stack() {
top = -1;
}
// Push an item onto the stack
void push(int item) {
if(top==9)
System.out.println("Stack is full.");
else
  stck[++top] = item;}
// Pop an item from the stack
int pop() {
if(top < 0) {
System.out.println("Stack underflow.");
return 0;
}else
return stck[top--];
}}
```

```
class TestStack {
public static void main(String args[])
{
Stack mystack1 = new Stack();
Stack mystack2 = new Stack();
// push some numbers onto the stack
for(int i=0; i<10; i++) mystack1.push(i);
for(int i=10; i<20; i++) mystack2.push(i);
// pop those numbers off the stack
System.out.println("Stack in mystack1:");
for(int i=0; i<10; i++)
System.out.println(mystack1.pop());
System.out.println("Stack in mystack2:");
for(int i=0; i<10; i++)
System.out.println(mystack2.pop());
}
}
```

Java Constructor

- **Difference between constructor and method in Java**

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as the class name.

Java Constructor

- **Java Copy Constructor**
- There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.
- There are many ways to copy the values of one object into another in Java. They are:
  - **By constructor**
  - **By assigning the values of one object into another**
  - **By clone() method of Object class**
- In this example, we are going to copy the values of one object into another using Java constructor.

Java Constructor

```
//Java program to initialize the values from one object to another object.
class Student6{
   int id;
   String name;
  //constructor to initialize integer and string
   Student6(int i,String n){
   id = i;
   name = n;
   }
   //constructor to initialize another object
   Student6(Student6 s){
   id = s.id;
   name =s.name;
   }
   void display(){System.out.println(id+" "+name);}
```

```
public static void main(String args[]){
   Student6 s1 = new Student6(111,"Karan");
   Student6 s2 = new Student6(s1);
   s1.display();
   s2.display();
  }
}
```

Output:

111 Karan

111 Karan

Java Constructor

- **Copying values without constructor**
- We can copy the values of one object into another by assigning the objects values to another object. In this case, there is no need to create the constructor.

```
class Student7{
  int id;
  String name;
  Student7(int i,String n){
  id = i;
  name = n;
  }
  Student7(){}
  void display()
    {System.out.println(id+" "+name);}
```

```
public static void main(String args[]){
   Student7 s1 = new Student7(111,"Karan
");
   Student7 s2 = new Student7();
   s2.id=s1.id;
   s2.name=s1.name;
   s1.display();
   s2.display();
  }
}
```

Output:

111 Karan

111 Karan

https://www.javatpoint.com/java-constructor